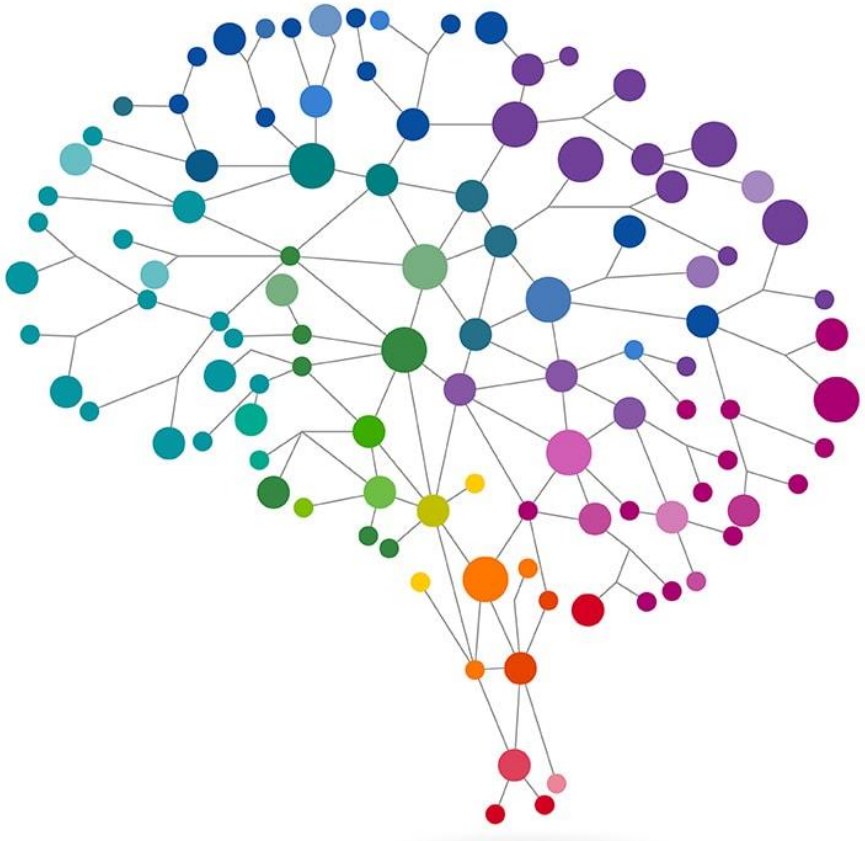
# Aprendizado de Máquina





Classroom code tg44fjf

Prof. Rodrigo Caropreso
Nov/2018
rcaropreso at gmail

# Conteúdo Programático – Aulas 1 e 2

## Introdução

- Conceitos Gerais em Aprendizado de Máquina
- Tipos de Aprendizado: Supervisionado, Semi-Supervisionado, Não-Supervisionado;
- Aprendizado por Reforço.

## Computação Evolutiva e Inteligência Coletiva

- Otimização por Enxame de Partículas (PSO).
- Algoritmos Genéticos;

## Redes Neurais Artificiais – Parte 1

- Algoritmo de Descida de Gradiente
- Perceptron Simples, Adaline;
- Lei de Aprendizado de Hebb;
- Regra Delta.

# Conteúdo Programático – Aulas 3 e 4

## Redes Neurais Artificiais – Parte 2

- Clusterização e K-Means;
- Classificação e K-NN;
- Redes de Base Radial;

## Análise Estatística

- Problemas de Classificação e Regressão;
- Regressão Linear e Logística;
- Parâmetros de Regularização;
- Naive Bayes.

# Algoritmos Genéticos



# Algoritmos Genéticos: Conceitos



- Faz parte da área conhecida como **Computação Evolutiva,** que consiste em um conjunto de modelos computacionais que tentam imitar a natureza evolutiva das espécies.
- Aplica-se normalmente em problemas relacionados à otimização de sistemas, em problemas de difícil manipulação pelas técnicas tradicionais.
- Fornece soluções ótimas para uma grande variedade de problemas:
  - Sequenciamento de procedimentos de produção.
  - Controle ótimo de processos.
  - Fluxo de potência em sistemas de energia

# Algoritmos Genéticos: Aplicações



## Otimização de Sistemas

- Sistemas não lineares, otimização restrita, otimização combinatorial, programação dinâmica, problemas de alocação de tarefas, seleção de rotas, aprendizagem de máquinas, etc.

## Processos de Busca

- Data mining (mineração de dados), investigação de conhecimento em bases de dados, indução de classificadores, etc.

## Arquiteturas Híbridas

- Integração com outros sistemas inteligentes visando otimização estrutural.
- Arquiteturas neuro-genéticas.
- Arquiteturas fuzzy-genéticas.
- Arquiteturas neuro-fuzzy-genéticas

# Algoritmos Genéticos: Problemas de Otimização



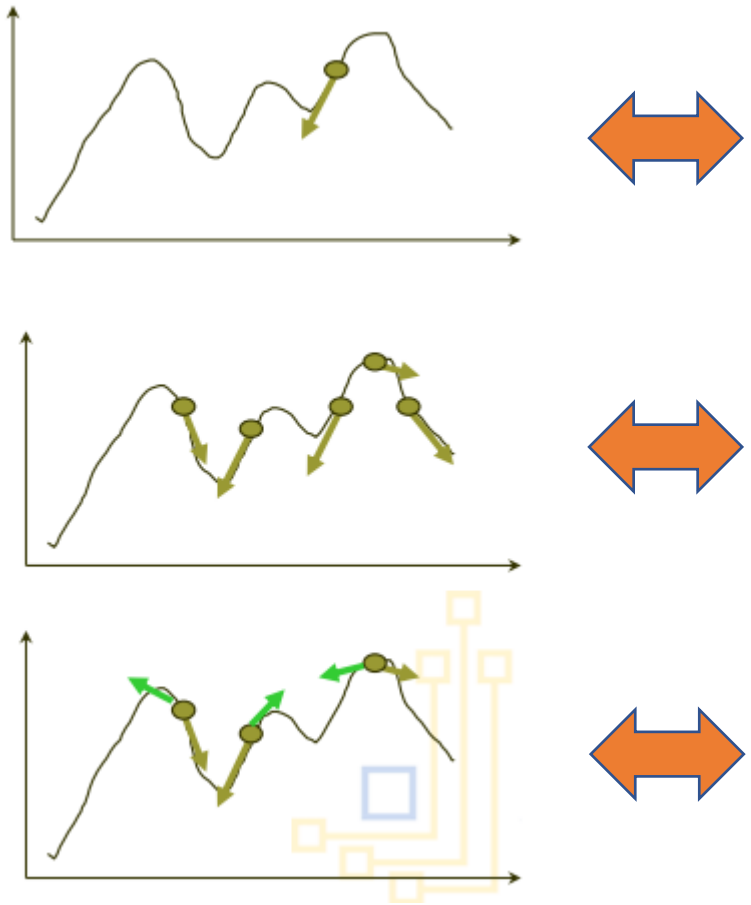
- Um problema típico de otimização consiste em:
  - Mapear matematicamente o sistema a partir de modelos físicos.
  - Definir a função custo ou função objetivo que se deseja maximizar ou minimizar.
  - Especificar o domínio em que as eventuais soluções estarão condicionadas.
  - Procurar (buscar) a melhor solução a fim de otimizar o sistema.

# Algoritmos Genéticos: Características Gerais



- Utilizam uma codificação do conjunto de parâmetros (indivíduos) e não com os próprios parâmetros (estados).
- Vasculham várias regiões do espaço de busca de cada vez.
- Utilizam informações diretas de qualidade, em contraste com as derivadas utilizadas nos métodos tradicionais de otimização.
- Utilizam regras de transição probabilísticas e não regras determinísticas

# Algoritmos Genéticos: Características Gerais



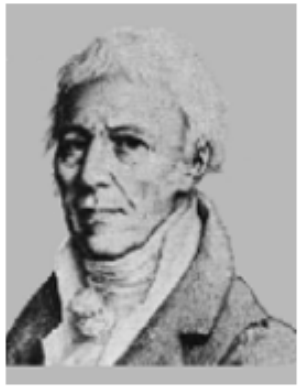
- **Métodos baseados em Gradiente**
  - Problema: mínimos locais.
- **Métodos Enumerativos**
  - Problema: Esforço computacional elevadíssimo.
- **Métodos Aleatórios**
  - Algoritmos Genéticos.
  - Indivíduos: representam diversas soluções potenciais.
  - Adaptativo e paralelo.

# Algoritmos Genéticos: Histórico

- Como explicar a diversidade de animais?
- Como explicar sua evolução?
- Qual é a influência dos antepassados?
- Qual é a influência do meio ambiente



- 1809: Jean-Baptiste Lamarck
  - **Lei do uso e do desuso**
    - Pelo uso e desuso de suas aptidões, a natureza força os seres a se adaptarem para sobreviverem.
  - **Lei dos caracteres adquiridos**
    - Os seres mais fortes são mais capazes de “transmitir” suas aptidões às novas gerações.

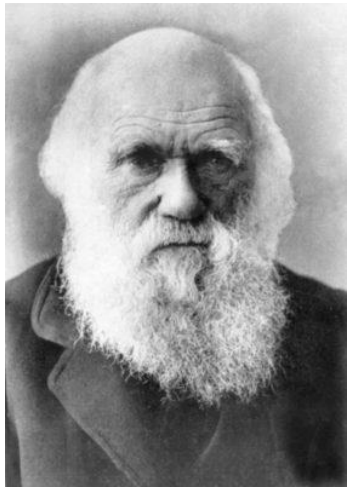


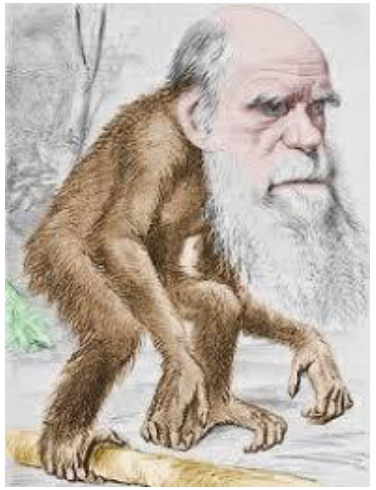


# Algoritmos Genéticos: Histórico

- 1859: Charles Darwin
  - Existe uma diversidade de seres devido aos contingentes da natureza (comida, clima, ...) e é pela lei da Seleção Natural que os seres mais adaptados aos seus ambientes sobrevivem.
  - Contra lei do uso de desuso.

- Os caracteres adquiridos são herdados pelas gerações seguintes (De onde vem o homem?)
- Teoria muito polêmica para o contexto da época.





# Algoritmos Genéticos: Histórico

- 1865: Gregor Mendel
  - Formalizou a “herança de características”, com a teoria do DNA (ervilhas)
- 1901: Hugo De Vries
  - Só a seleção natural não é responsável pela produção de novas (mais adaptadas) espécies. Tem de haver uma mudança genética!
  - Formalizou o processo de geração de diversidade: Teoria da Mutação (Contra lei do uso de desuso)
- 1975: John Holland
  - Idealizou os algoritmos genéticos (Adaptation in Natural & Artificial Systems, MIT Press, 1975 (2nd ed. 1992) ).







# Algoritmos Genéticos: Taxonomia Fundamental



- **Genes**
  - Codificam uma característica particular.
- **Cromossomo**
  - Codificação de uma possível solução (a solução é um “Indivíduo”).
- **Indivíduos**
  - Conjunto de atributos da solução (concatenação das sequências de bits).
- **População**
  - Conjunto de indivíduos sendo cogitados como solução.
  - Populações pequenas têm grandes chances de perder a diversidade necessária (exploração de todo o espaço de soluções).
  - Populações grandes perderão grande parte de sua eficiência pela demora em avaliar a função de custo (*fitness*).

# Algoritmos Genéticos: Taxonomia Fundamental



- **Reprodução**
  - **Reprodução Sexuada**
    - Genes são intercambiados entre dois pais (crossover).
    - Os filhos estão sujeitos a modificações, na qual bits elementares são mudados (mutação).
  - **Reprodução Assexuada**
    - O filhos são duplicações dos genes dos pais.
- **Função de Custo ou Aptidão (Fitness)**
  - Mede a adaptação do indivíduo ou quão boa é a solução dada por este indivíduo.
  - Deve ser representativa do problema, diferenciando uma solução boa daquela má.
  - Heurística de busca no espaço de estado.

# Algoritmos Genéticos: Requisitos



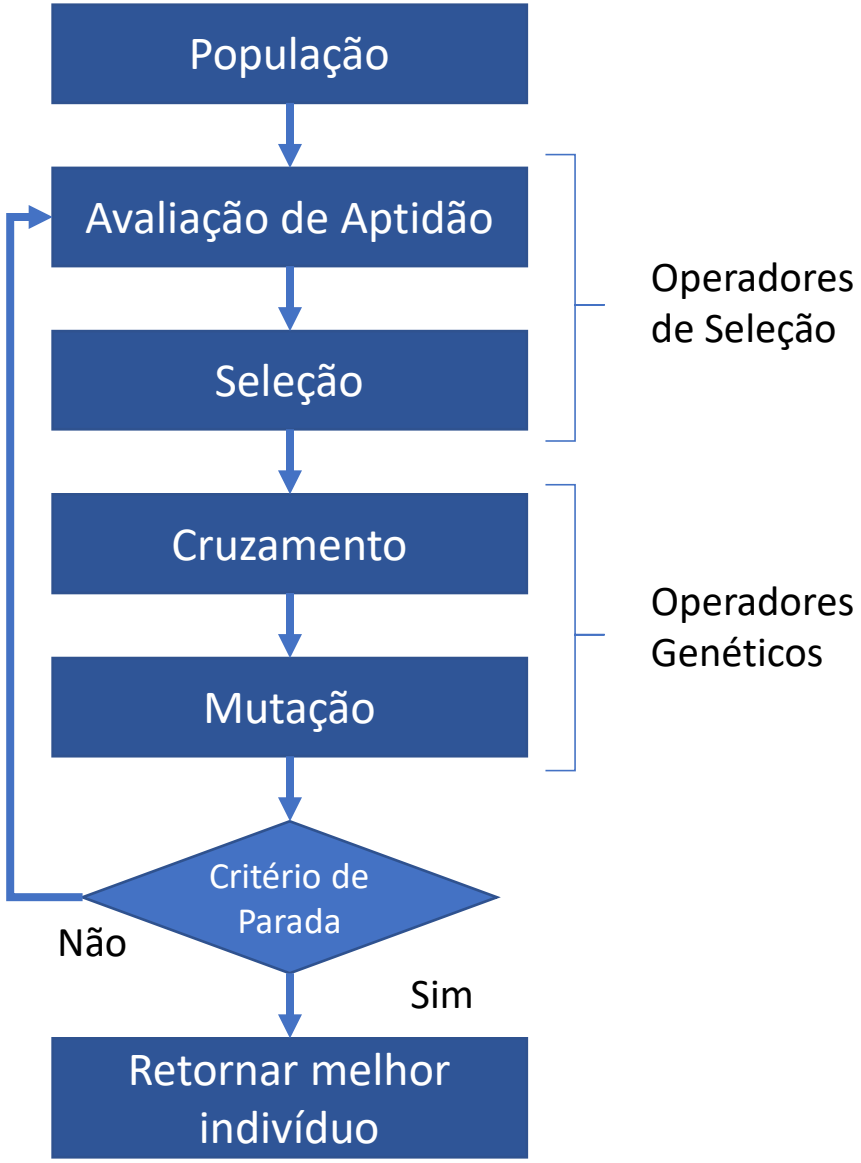
- Representações das possíveis soluções do problema no formato de um código genético.
- População inicial (aleatória) que contenha diversidade suficiente para permitir ao algoritmo combinar características e produzir novas soluções (Ex: n=100).
- Existência de um método para medir a qualidade (Função Fitness) de uma solução potencial.
- Especificação de um procedimento de combinação de soluções para gerar novos indivíduos na população.

# Algoritmos Genéticos: Requisitos



- Especificação de um critério de escolha das soluções que permanecerão na população ou que serão retirados desta.
- Um procedimento para introduzir periodicamente alterações em algumas soluções da população, visando sobretudo:
  - Manter a diversidade da população.
  - Possibilitar a produção de soluções inovadoras para serem avaliadas pelo critério de seleção dos mais aptos.

# Algoritmos Genéticos: Estrutura Funcional




1. Gerar População Inicial;
2. Descartar uma parte dos Indivíduos menos aptos;
3. Aplicar operadores de reprodução;
4. Aplicar operadores de mutação;
5. Se o critério de parada foi satisfeito

   Então → encerrar processo.

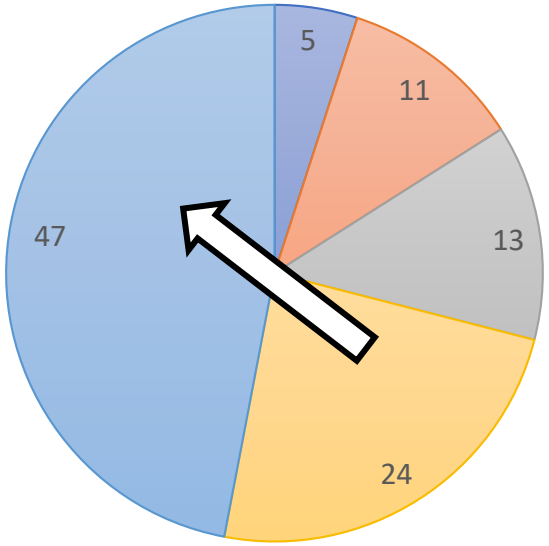
   Senão → voltar ao passo 2.

# Algoritmos Genéticos: Seleção de indivíduos

| Indivíduo | Apt. Relativa | Apt. Absoluta |
|---|---|---|
| 1 | 2 | 0,0526 (5%) |
| 2 | 4 | 0,1053 (11%) |
| 3 | 5 | 0,1316 (13%) |
| 4 | 9 | 0,2368 (24%) |
| 5 | 18 | 0,4737 (47%) |
| Total | 38 | 1 (100%) |

- Utiliza uma função de avaliação (função de fitness) para medir a aptidão de cada indivíduo.
- Essa aptidão pode ser absoluta ou relativa.
- Métodos de métodos de seleção mais comuns:
  - Roleta.
  - Torneio.
  - Amostragem Universal Estocástica.

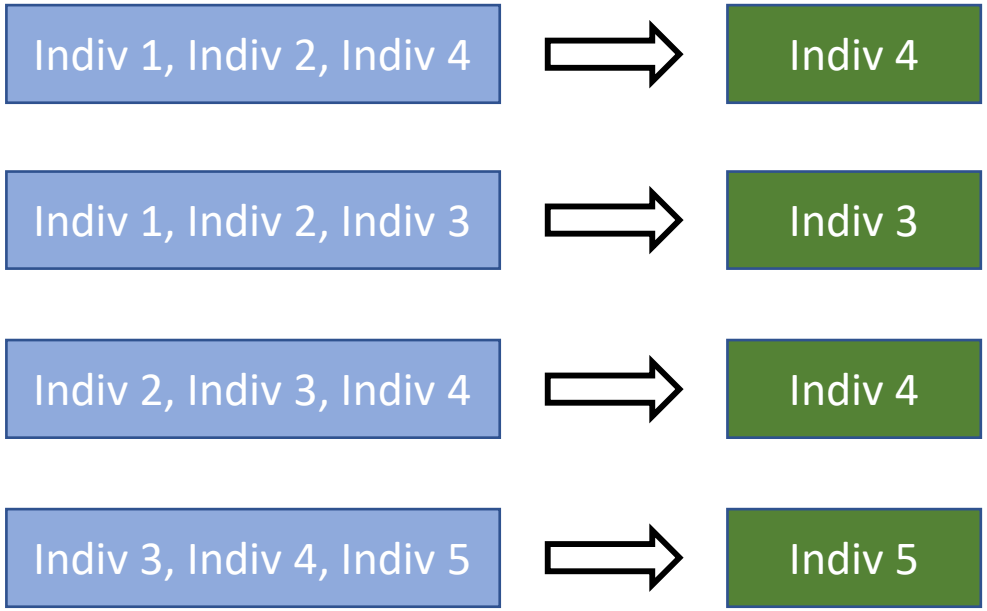
# Algoritmos Genéticos: Seleção de indivíduos (Roleta)




| 1 | 5% |
|---|---|
| 2 | 11% |
| 3 | 13% |
| 4 | 24% |
| 5 | 47% |

- Coloca-se os indivíduos em uma roleta, dando a cada uma “fatia” proporcional à sua aptidão relativa.
- Depois roda-se a agulha da roleta. O indivíduo em cuja fatia a agulha parar permanece para a próxima geração.
- Repete-se o sorteio quantas vezes forem necessárias para selecionar a quantidade desejada de indivíduos.

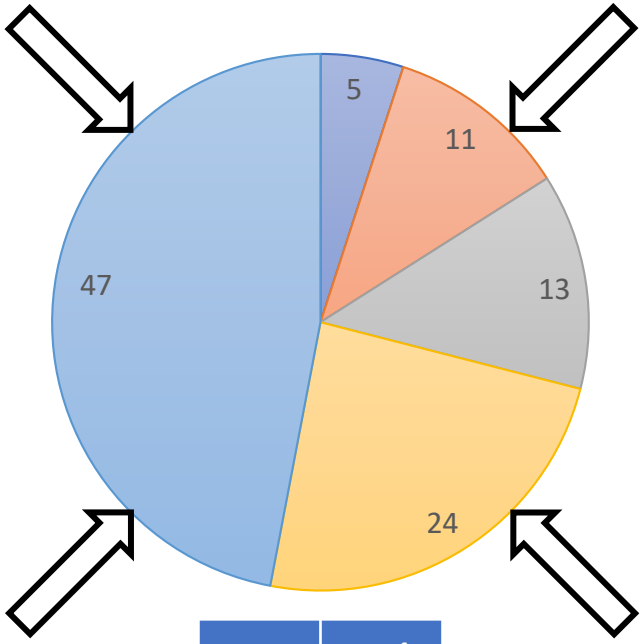
# Algoritmos Genéticos: Seleção de indivíduos (Torneio)




| 1 | 5% |
|---|---|
| 2 | 11% |
| 3 | 13% |
| 4 | 24% |
| 5 | 47% |

- Utiliza sucessivas disputas para realizar a seleção.
- Para selecionar k indivíduos, são realizadas k disputas, onde cada disputa envolve n indivíduos escolhidos ao acaso.
- O indivíduo de maior aptidão na disputa é selecionado.
- É muito comum utilizar n = 3.

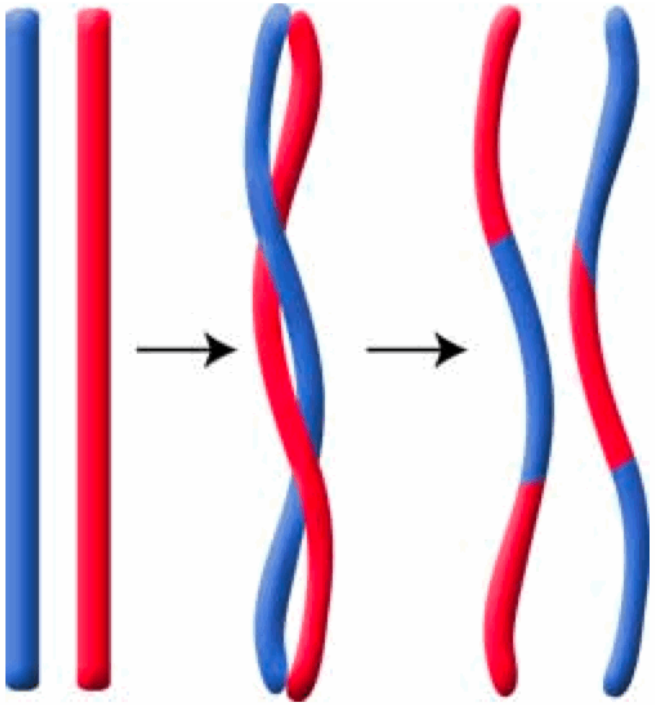
# Algoritmos Genéticos: Seleção de indivíduos (Amostragem Univ. Estocástica)




| 1 | 5% |
|---|---|
| 2 | 11% |
| 3 | 13% |
| 4 | 24% |
| 5 | 47% |

- SUS – Stochastic Universal Sampling.
- Semelhante à Roleta, mas para selecionar k indivíduos utiliza k agulhas igualmente espaçadas, girando-as em conjunto uma só vez.
- Apresenta resultados menos variantes que a Roleta.

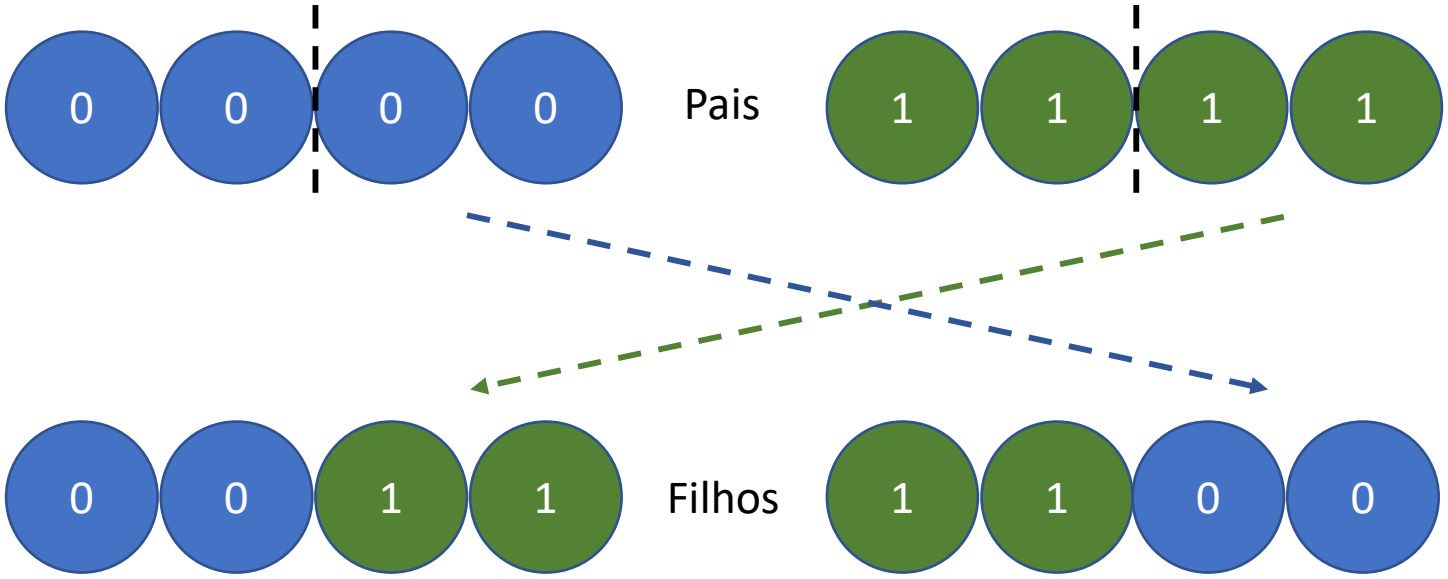
# Algoritmos Genéticos: Operador de Cruzamento (Reprodução)



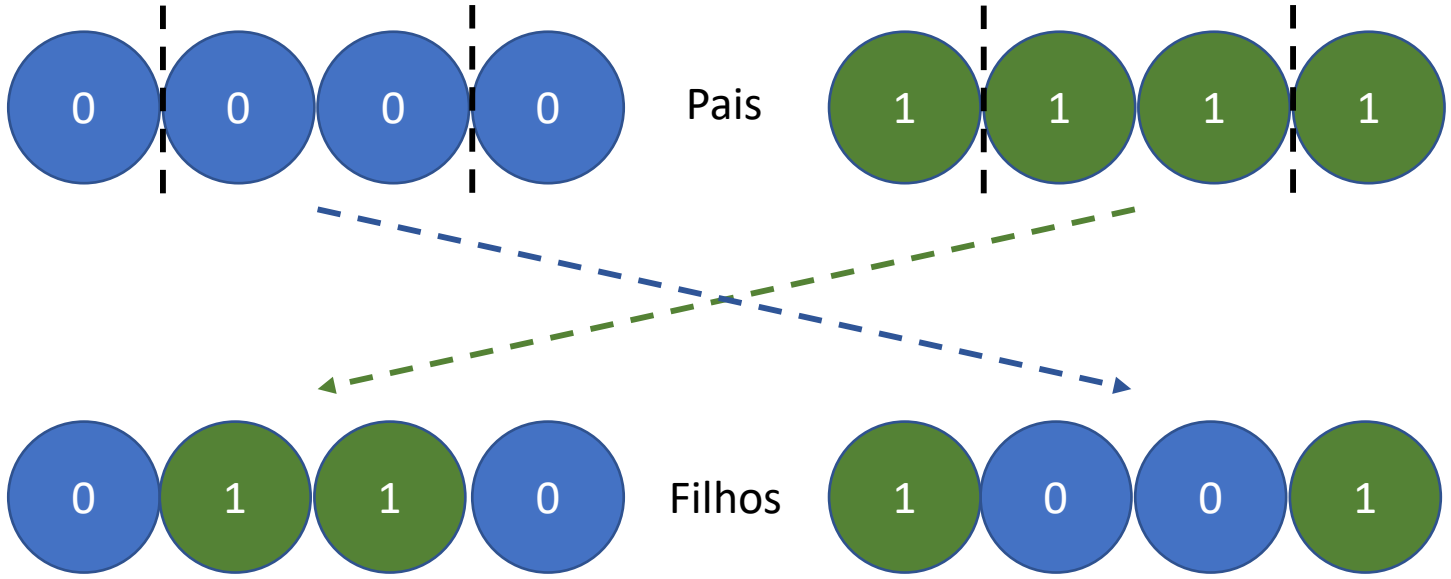
http://www.genomenewsnetwork.org/resources/whats_a_genome/Chp3_2.shtml

- Operador genético principal.
- Combina as informações genéticas de dois indivíduos (pais) para gerar novos indivíduos (filhos).
- Versões mais comuns criam sempre dois filhos para cada operação.
- Responsável por gerar novos indivíduos diferentes (sejam melhores ou piores) a partir de indivíduos já promissores.
- Aplicado a cada par de indivíduos com alta probabilidade (normalmente entre 0,6 e 0,99).

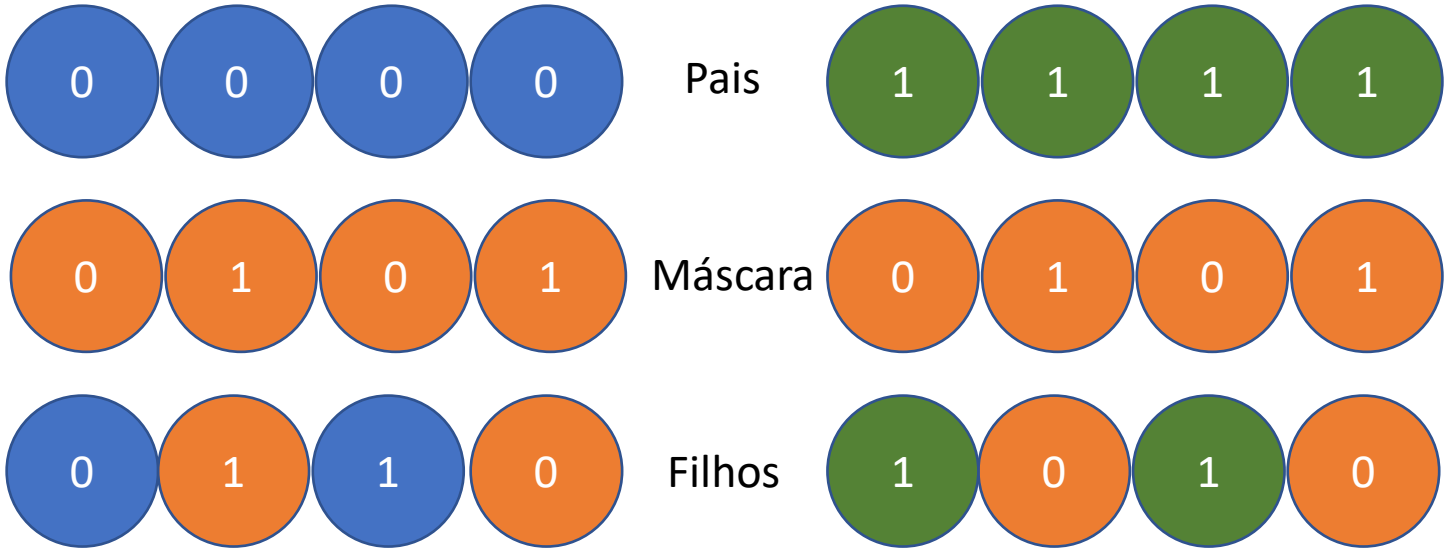
# Algoritmos Genéticos: Operador de Cruzamento - Um ponto



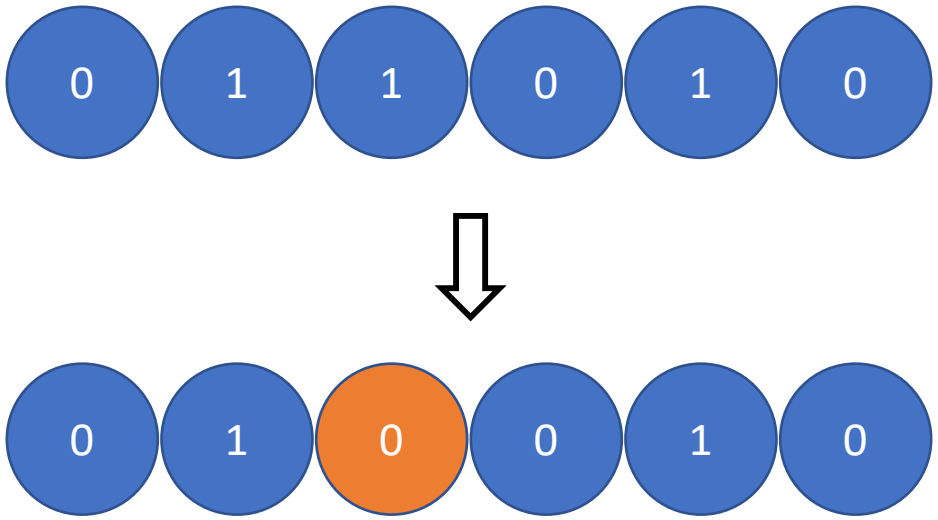
# Algoritmos Genéticos: Operador de Cruzamento – Multi-ponto



# Algoritmos Genéticos: Operador de Cruzamento - Uniforme

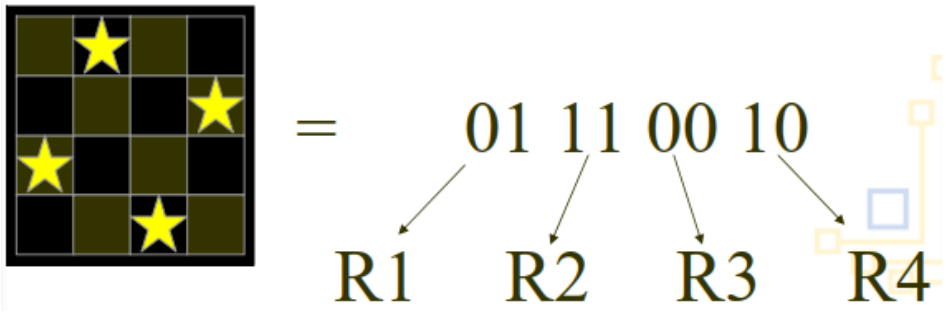


# Algoritmos Genéticos: Operador de Mutação




- Operador randômico de manipulação.
- Introduz e mantém a variedade genética (diversidade) da população.
- Garante a possibilidade de se alcançar qualquer ponto do espaço de busca.
- Cria novas características que não existiam.
- Escapam e contornam mínimos locais.
- É um operador genético secundário.
- Se seu uso for exagerado, reduz a evolução à uma busca totalmente aleatória.
- Assim, indivíduos sofrem mutações com probabilidade baixa:
  - Taxa de mutação → entre 0,1% e 1% dos indivíduos da população selecionada.
  - A taxa pode ir diminuindo para garantir convergência da solução

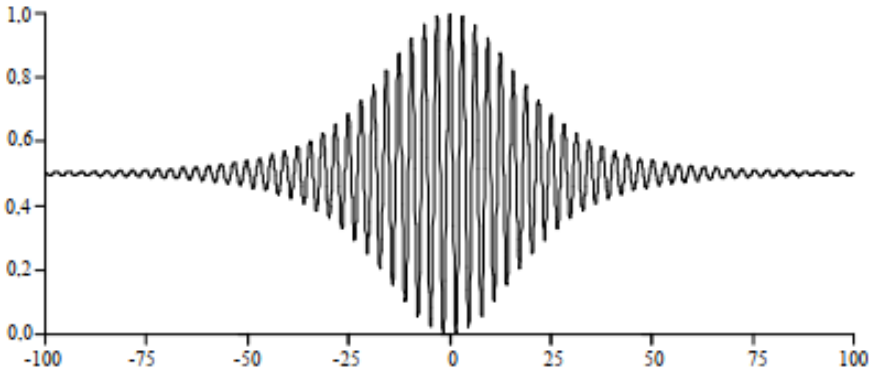
# Algoritmos Genéticos: Representação de Soluções




Exemplo para N = 4

- Variáveis Discretas (Enumeradas):
- Problemas de otimização combinatorial, programação dinâmica, etc.
- Exemplo: Problema das N-Rainhas
- A posição de cada rainha é dada por uma subcadeia do cromossomo

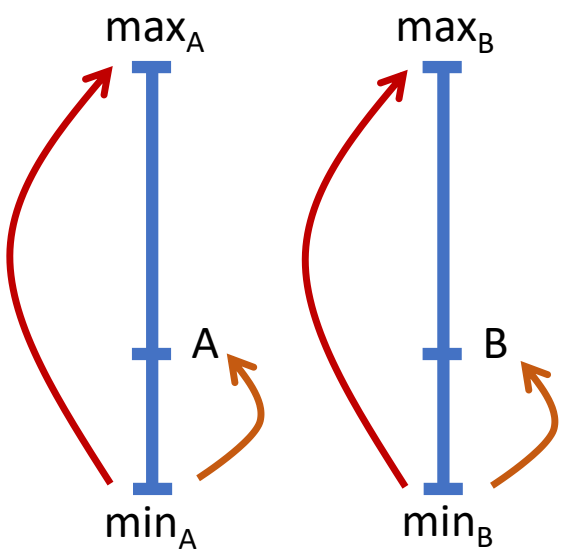
# Algoritmos Genéticos: Representação de Soluções




- Variáveis Contínuas (Reais):
  - Problemas de otimização não linear.
- Exemplo: Minimizar a seguinte função:

$$f(x,y) = 0{,}5 - \frac{\left(sen\sqrt{x^2+y^2}\right)^2 - 0{,}5}{\left(1{,}0 + 0{,}001(x^2+y^2)\right)^2}$$

# Algoritmos Genéticos: Transformação de Domínios

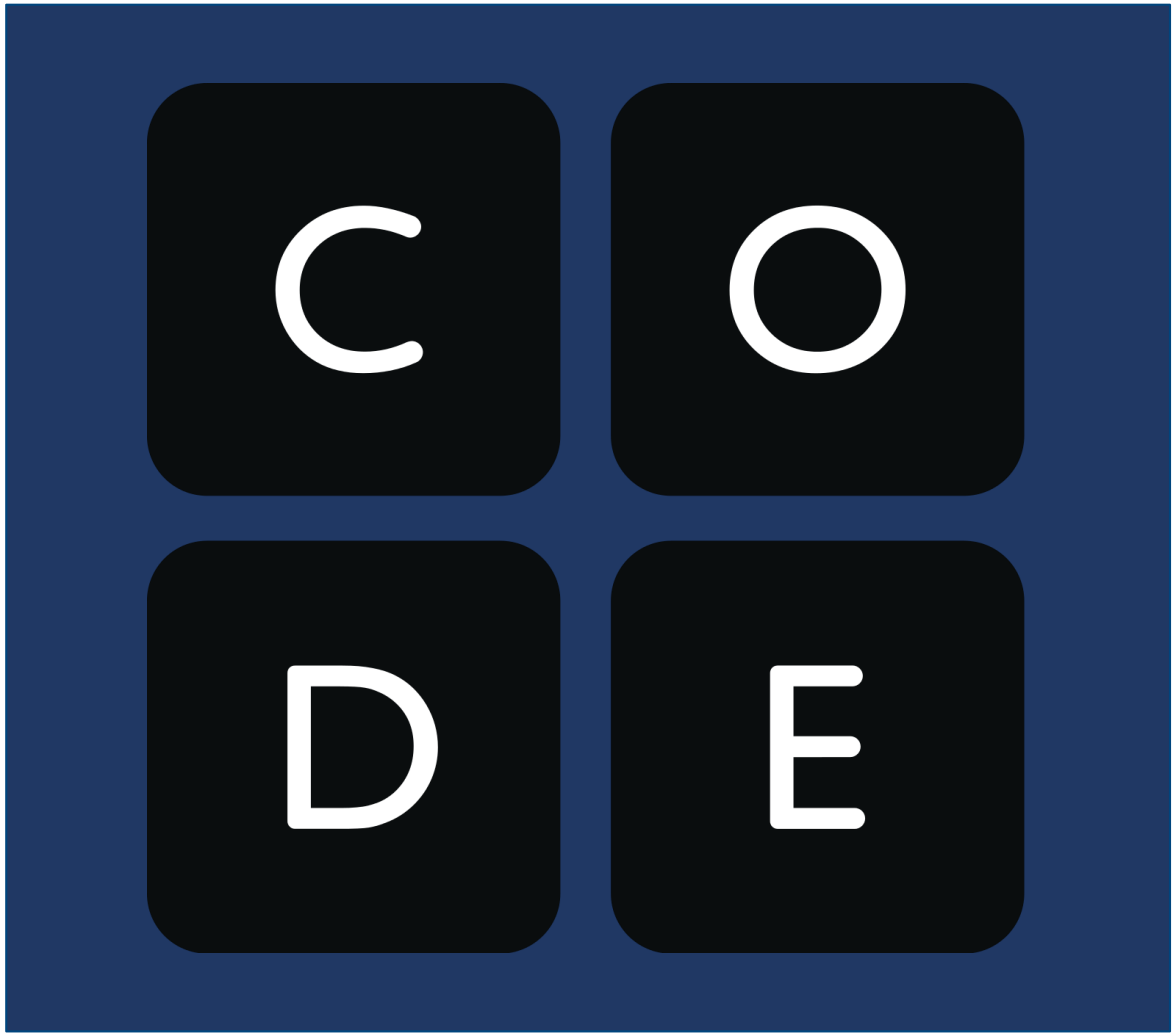



Teorema de Tales

- É necessária para realizar a adequação dos extremos do domínio do problema aos valores a serem utilizados pela população-solução.
- É bastante comum o uso do Teorema de Tales para normalização de domínio:

$$\frac{A - Min_A}{Max_A - Min_A} = \frac{B - Min_B}{Max_B - Min_B}$$

## Challenges Time






# Laboratório 1: Transformação de Domínios/Função de Aptidão




- **Utilizando Jupyter e NumPy**: para a função de variável contínua apresentada, faça o mapeamento de domínio e decodifique a cadeia de bits abaixo (22 bits) para o intervalo [-1, 2]. Plote a função abaixo.
- Notebook: facens_ml_Aula1_AG_Lab01

$$f(x) = x \cdot sen(10 \cdot \pi \cdot x) + 1$$

# Laboratório 2: Algoritmo de Seleção Natural




- **Utilizando Jupyter e NumPy:** implemente o algoritmo de seleção por roleta para a Tabela abaixo.
- Notebook: facens_ml_Aula1_AG_Lab02

| Indivíduo | Apt. Relativa | Apt. Absoluta |
|---|---|---|
| 1 | 2 | 0,0526 (5%) |
| 2 | 4 | 0,1053 (11%) |
| 3 | 5 | 0,1316 (13%) |
| 4 | 9 | 0,2368 (24%) |
| 5 | 18 | 0,4737 (47%) |
| Total | 38 | 1 (100%) |

# Laboratório 2: Algoritmo de Seleção Natural




- **Utilizando Jupyter e NumPy:** implemente o algoritmo de seleção por torneio para a Função contínua dada na atividade anterior, considerando o domínio dado.
- Notebook: facens_ml_Aula1_AG_Lab02

# Laboratório 3: Seleção Natural




- **Utilizando Jupyter e NumPy:** complete a função de seleção natural com auxílio do professor.
- Notebook: facens_ml_Aula1_AG_Lab03

# Laboratório 4: Operador de Crossover




- **Utilizando Jupyter e NumPy:** implemente o algoritmo de reprodução por crossing over (1 Ponto) para a Função contínua dada na atividade anterior, considerando o domínio dado.

- Notebook: facens_ml_Aula1_AG_Lab04

# Laboratório 4: Operador de Crossover




- **Utilizando Jupyter e NumPy:** implemente o algoritmo de reprodução por crossing over (2 Pontos) para a Função contínua dada na atividade anterior, considerando o domínio dado.

- Notebook: facens_ml_Aula1_AG_Lab05

# Laboratório 5: Operador de Mutação




- **Utilizando Jupyter e NumPy:** implemente o algoritmo de mutação para a Função contínua dada na atividade anterior, considerando o domínio dado.

# Laboratório 6: Algoritmo Genético




- **Utilizando Jupyter e NumPy:** implemente um algoritmo para criar aleatoriamente uma população de 100 indivíduos de acordo com a função dada e, utilizando as funções criadas nos labs anteriores, construa o algoritmo genético de otimização para a função contínua dada.

# Referências





- https://www.coursera.org/
- https://python.org.br/
- http://scikit-learn.org/stable/
- https://www.w3schools.com/python
- https://www.gnu.org/software/octave/
- https://octave-online.net/
- https://www.mathworks.com/products/matlab.html

## Referências





- RUSSELL, STUART J.. Inteligência Artificial. 2ª ed. São Paulo: Campus - Elsevier, 2004.
- FERNANDES, Anita M. R.. Inteligência Artificial: noções gerais. 1ª ed. Florianópolis: Visual Books, 2008. Capítulo 1.
- SILVA, I.N. – Sistemas Inteligentes – Notas de Aula da disciplina de Sistemas Inteligentes – USP São Carlos, 2012.
- GOZALEZ, J.A.Q., ZAMPIROLLI, F.A. – Mineração de Dados e Regras de Associação. Notas de Aula da disciplina Sistemas Inteligentes. CMCC – UFABC, 2013.



# Facens

AQUI TEM ENGENHARIA